

Converge

The Eight Passes — Compile Intent, Close the Loop

The canonical blueprint for compiling a fuzzy brief into an autonomous, eval-gated system — eight passes, a fork, and a closed loop that drives a fleet to green. Build the factory from day one; walk out one gate at a time.

Document type	Canonical process blueprint — the repeatable steps, v2
New in v2	Pass 8 · The Loop · the execution loop (task-loop) with the Manager as a future CI/CD pass · the tracker as state · the two moats
Worked example	postgres → duckdb → dbt → MCP (this repo), task-driven
The invariant	Every pass lowers altitude, binds an engine, ends at a gate — except the Loop, which closes
Converged means	An eval passed — not that you feel done

Loop Engineering — the discipline of designing the loops that prompt agents, rather than prompting agents — was named in June 2026. This document places it where it belongs: as the eighth pass that closes the spine.

Introduction

Why Converge exists

The old loop ran in your head. A requirement arrived fuzzy, and you held the definition of "done" as private judgment — refined by taste, defended in review, never written down. That loop does not scale to AI engines, because an engine cannot execute a standard it cannot read. The bottleneck was never typing speed; it was that intent lived in a human and had to be re-supervised at every step. The AI-native engineer inverts this: take a fuzzy requirement and lower it through a repeatable descent — each step more precise than the last — until "done" is a machine-checkable condition an engine can satisfy unsupervised.

The main idea — compile intent, don't write the system

Stop writing the system. Compile it. Converge treats building as a compilation pipeline that lowers altitude in fixed stages: intent → structure → plan → tasks → code → loop. Each stage is a transformation with a typed output and a gate that must pass before the next stage runs — exactly how a compiler lowers source through intermediate representations down to machine code. You author the high-level intent and the checks; the pipeline emits the system. Your role shifts from author to compiler — you specify the descent and verify each lowering, rather than hand-writing the final artifact. The discipline is the compiler; the engine is the backend.

The thesis — build the factory from day one

This is the document's central claim: you do not transform a workflow into a factory later. That retrofit — bolting evals, harnesses, and loops onto human-centric code never built to be gated — is the slow, expensive path, and it is the path almost everyone takes. Converge refuses it. The framework is factory-shaped from Pass 1: every pass already ends in a gate, every task is born with a runnable eval, and the harness is a control plane from the first file. Nothing here is added to "make it a factory" afterward — the factory is the scaffold you build on from the start. So when you decide to step out, there is nothing to retrofit and nothing to rewrite; the gates already hold without you, and you walk out one gate at a time. Modular in use — take any single pass on its own — but factory-shaped in design. You are not automating a workflow you already have; you are building, from the first pass, the thing that was always meant to run without you.

EVERY PASS

Lowers altitude

intent → structure → plan →
tasks → code → loop — each
step closer to something a
machine runs.

BINDS

An engine

the right tool for the altitude —
CoWork to think, Code to build,
an adversary to attack, a fleet to
crank.

ENDS AT

A gate

converged means an eval
passed — not that you feel done.
The gate is the unit of trust.

THE INVARIANT

Every pass lowers altitude, binds an engine, ends at a gate. **Pass 8 is the exception — it does not lower, it closes.** You are converged when the eval passes — not when you feel done.

Contents

—	Introduction — What Converge Is, and Why
0.	The Whole Method, On One Line
—	The Spine, Drawn — the 8-pass descent
1.	Intent — brd-docs-to-tech-req
2.	Structure — tech-req-to-adrs (ADRs)
3.	Decompose — reqs-to-swimlane-plans
4.	Consensus — sketch-plans-adversarial-review
—	The Fork — Plan-Driven (HITL) vs Task-Driven (walk out)
5A.	Specify — plans-to-coherent-spec
5B.	Tasking — task-spec
①	Register — task-specs-to-issues (the tracker as state)
6.	Harness — stack-to-harness (the quality moat)
8.	The Loop — task-loop (the execution loop); Manager = future CI/CD
—	Why This Is Defensible — the two moats, positioning, the honest boundary
—	One Method, Every Engineer
—	The Method, End to End on One Feature (this repo)

0 • The Whole Method, On One Line

Intent -> Structure -> Decompose -> Consensus -> [Fork: Specify | Tasking] -> Harness -> Execute -> Loop.
A fuzzy brief becomes a tech-spec, the tech-spec becomes ADRs, the ADRs become swimlane plans, the plans survive an adversarial review, and there the path forks on where the trust boundary sits — Specify wraps the whole system, Tasking wraps each unit. Both branches then run through the same Harness, Execute, and Loop. Each output is the next input; the chain descends from a fuzzy brief to a fleet running green.

#	PASS	SKILL	OUT / GATE
1	Intent	brd-docs-to-tech-req	tech-spec · answers the brief
2	Structure	tech-req-to-adrs	docs/adrs/* · system understood
3	Decompose	reqs-to-swimlane-plans	swimlane plans · plan-altitude only
4	Consensus	sketch-plans-adversarial-review	plans sharpened · fork decided
■	The Fork	– decision –	where does the trust boundary sit?
5A	Specify	plans-to-coherent-spec	one coherent spec · one e2e eval
5B	Tasking	task-spec	many units · each has an eval
①	Register	task-specs-to-issues	1 spec = 1 issue · the board is state
6	Harness	stack-to-harness	.claude/ + AGENTS.md · control plane
8	The Loop	task-loop (--issue N)	each task → eval green → PR

THE INVARIANT

Every pass lowers altitude, binds an engine, ends at a gate. Three guardrails make the sequence **canonical**, not merely descriptive: **Pass 2 writes ADRs only** (no code), **Pass 3 is plan-altitude only**, and **the fork is an output of Pass 4** — never assumed earlier. And Pass 8 does not lower — it closes.

The spine, drawn — eight passes, one fork

The full descent as a flow: four passes lower altitude in a straight line, the path forks at Consensus into 5A · Specify and 5B · Tasking, and both branches reconverge through Harness, Execute, and the Loop.

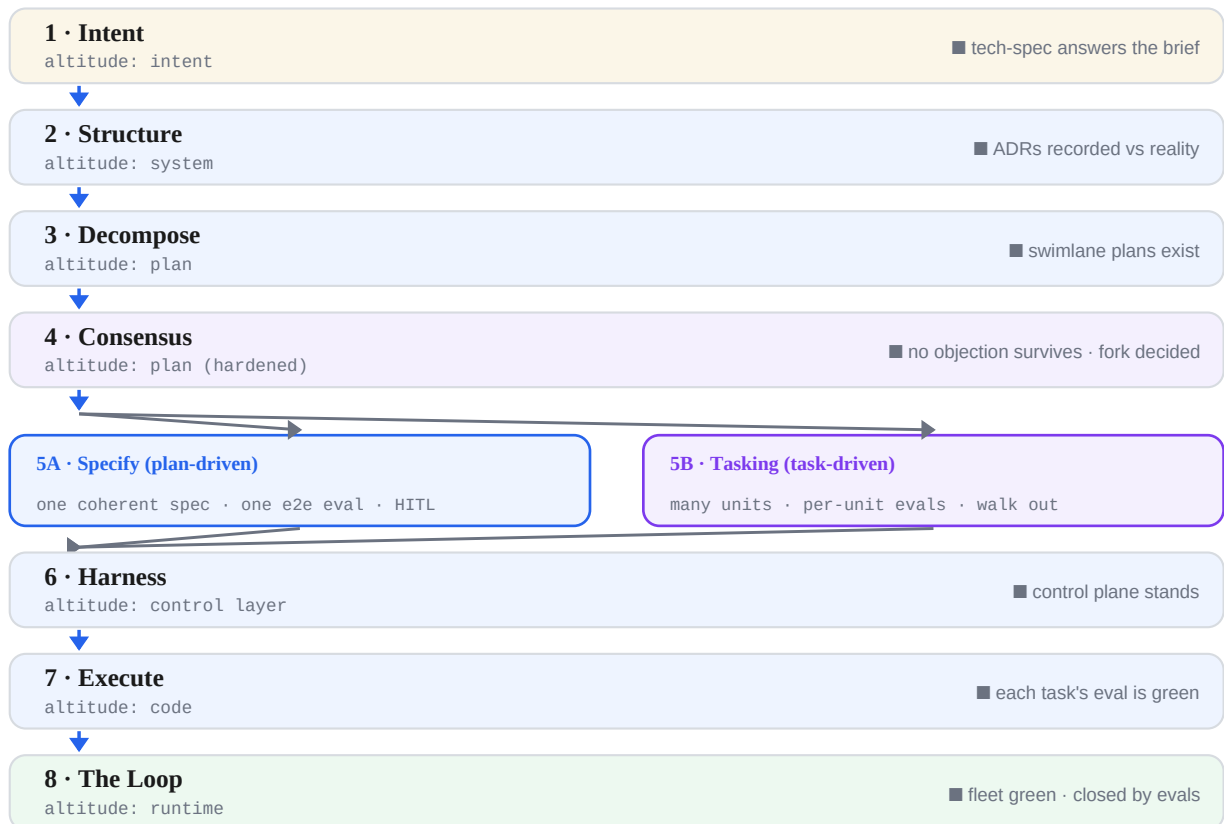


FIG 1 The 8-pass descent — each pass ends at its gate. After Consensus the path forks (5A Specify / 5B Tasking) and reconverges at Harness; Pass 8 closes the loop.

The client hands you the problem. You produce the spec.

ENGINE · CLAUDE COWORK

SKILL · brd-docs-to-tech-req

OUT · TECH-SPEC

Read the BRD like a senior engineer, not a stenographer. Find the real problem underneath the ask, name who actually hurts when it is unsolved, and pin the numbers that define success.

UNDERSTAND

Read

the BRD like a senior engineer — the real problem underneath, who hurts, the numbers that define success.

INTERROGATE

Grill

the 2–3 questions whose answers most change how this gets built; ambiguity here compounds downstream.

CRYSTALLIZE

Spec

resolve the open questions into one technical answer — the tech-spec that says what gets built and how success is measured.

GATE

You can restate the client's problem in one breath and show the tech-spec answers it — scope, data, and success metric named. The tech-spec is the artifact; clearing the gate is the judgment that it truly answers the brief.

Greenfield or brownfield — and write the ADRs.

ENGINE · CLAUDE CODE · ON THE REPO

SKILL · tech-req-to-adr

OUT · docs/adrs/*.md

Before any plan, name the ground. BROWNFIELD is a system you did not write: open it, explain it end to end, and check the spec against the real schemas, sources, and jobs you find. GREENFIELD is a system that does not yet exist: confirm there is nothing to inherit, then ground the spec in the source systems and constraints it must honor. Either way you leave Pass 2 knowing the terrain — what is actually there, and what the work must respect.

NAME THE GROUND**Field**

brownfield (a system you didn't write) vs greenfield (nothing yet) — name it explicitly; the rest follows.

GROUND**Check**

open the system end to end; reconcile the spec against real schemas, sources, and jobs.

RECORD**ADRs**

write docs/adrs/*.md — grounding decisions (facts + constraints), never how to build. That's Pass 3.

GATE

The system is understood and its grounding decisions are recorded durably, so Pass 3 can read them without re-deriving the terrain. An ADR records what is true — schemas, joins, constraints — never how to build. The moment an ADR says "build X", you've drifted into planning.

Split the work along its natural seams.

ENGINE · CLAUDE CODE (AUTO) · SAME SESSION

SKILL · reqs-to-swimlane-plans

IN · READS THE ADRs

takes the system understood in Pass 2 — same session, ADRs in hand — and splits the work along its natural seams. Read the ADRs, then cut where the system already wants to be cut: by feature or by component. Each seam becomes a swimlane; each swimlane gets its own focus-oriented sketch plan. The common case is two lanes — transform versus serve, a medallion lane and a serving lane — but a system carrying several features splits into several lanes. The right number of lanes is the number of genuine seams. No more, no fewer.

DECOMPOSE

Seams

cut where the system is already jointed — a feature edge or a component edge, not an arbitrary slice.

SWIMLANE

One plan each

every seam becomes one swimlane carrying exactly one sketch plan — one lane, one plan, one focus.

GROUNDING

Reads ADRs

each lane inherits the architecture decisions recorded in docs/adrs/ — it builds on Pass 2, not around it.

GATE

One sketch plan per swimlane, each listing features, dependencies, and a sane build order, inheriting the ADR decisions. Plan-altitude only — no atomic tasks, no code. The session carries from Pass 2; the understanding becomes the plans without leaving the session.

Reference architecture — the common transform → serve case

One concrete instance of the seams Pass 3 splits into swimlanes: a transformation lane (medallion: raw → bronze → silver → gold) feeding a serving lane (MCP / API over gold). Other systems split by feature or component — this is the shape, not the rule.

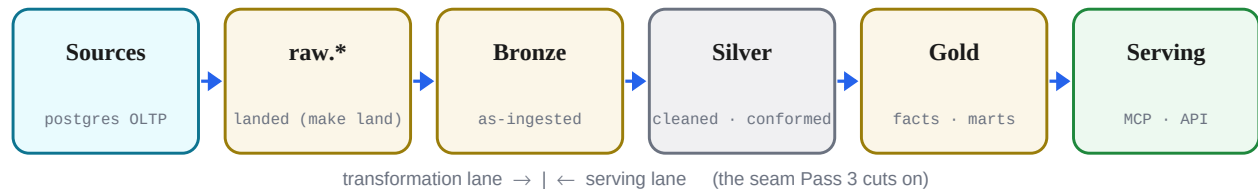


FIG 2 Reference architecture on this repo — postgres sources land to raw.*, the medallion lane transforms to gold, the serving lane exposes it via MCP. Pass 3 cuts the plans on the seam between the lanes.

Bring a different engine to refute the plans.

ENGINE · ADVERSARY (--adversary codex)

SKILL · sketch-plans-adversarial-review

OUT · PLANS SHARPENED + FORK DECIDED

here is the body text for that section, ready to drop into the PDF.

ATTACK

Refute
a different model, one plan at a time, as a skeptic who didn't write it — default to refuted.

GROUND

Drift?
check each plan against the tech-spec and the ADRs — contradictions, gaps, silent assumptions.

SHARPEN

Fix / Own
every objection FIXED in a plan or ACCEPTED with a named owner — nothing silently dropped.

GATE

No open objection survives, and the FORK is decided here as an output of this pass — does the trust boundary wrap the whole system (Fork A · plan-driven) or each unit (Fork B · task-driven)? Cross-model disagreement is the point: a model won't refute itself hard enough.

One big gate, or many small ones?

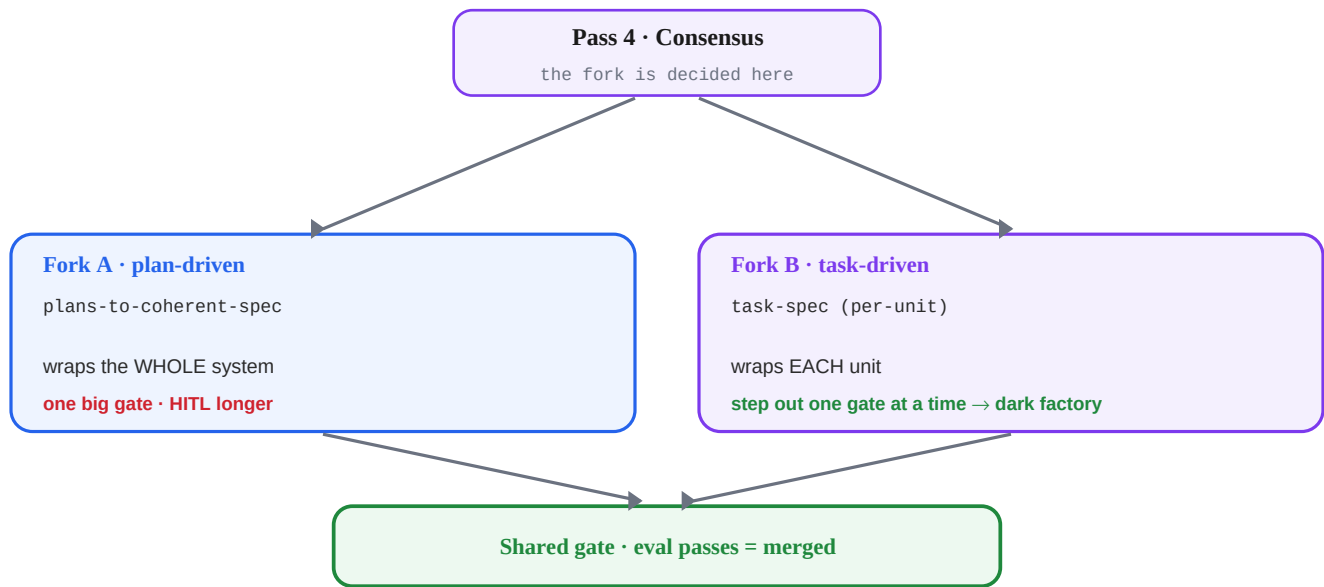


FIG 3 The fork branches at Consensus into two paths — Fork A holds one big human gate, Fork B lets you step out one small gate at a time — and both reconverge on the identical shared gate.

Pass 4 ends by deciding one thing: where does the trust boundary sit? Every pass before it lowered altitude in a straight line. Here the line splits. The question is not which framework you like — it is how large a unit of work you are willing to trust at once. Both answers are valid Converge. Both reconverge on the identical gate. But they differ in one consequence that decides everything downstream: how long a human stays in the loop, and how you get out.

FORK A · PLAN-DRIVEN (skill: plans-to-coherent-spec). The reviewed swimlane plans fold into ONE coherent, coupled spec — the named SDD frameworks live here (SpecKit, Kiro, OpenSpec, BMAD). The trust boundary wraps the WHOLE system: the spec is one object, verified by ONE end-to-end eval. Because the unit of trust is large and internally coupled, you cannot step out piecemeal — nothing is "done" until everything passes together. So this path keeps a HUMAN AT ONE BIG GATE, and keeps them there longer. The human reviews the whole coherent spec, holds the single gate, and releases it as a unit. It is the right shape when the system genuinely resists decomposition — when the parts only make sense together.

FORK B · TASK-DRIVEN. The plans decompose into MANY atomic, eval-backed units — task-specs (skill: task-spec). The trust boundary wraps EACH unit, not the system. Verification is per-unit: every task carries its own runnable eval. Because the gates are granular, trust is earned one unit at a time — each eval that goes green is a gate you no longer have to hold by hand. You STEP OUT ONE GATE AT A TIME as each unit proves itself. This is the path that REACHES THE DARK FACTORY: many small, automatable gates are exactly what a fleet of workers can close on a schedule without you standing over each one.

The mechanism behind the difference is GRANULARITY of the trust boundary. Fork A is one big gate a human holds; Fork B is many small gates you can automate one at a time. Both reconverge on the identical

rule — eval passes = merged — and that shared gate is what makes Converge ONE method with two paths, not two methods. Either path can go dark-factory; only Fork B gets there incrementally, removing the human one gate at a time instead of all at once.

	FORK A · PLAN-DRIVEN	FORK B · TASK-DRIVEN
framework	plans-to-coherent-spec — Speckit / Kiro / OpenSpec / BMAD	task-spec — per-unit
coupling	one coherent, coupled spec	many atomic, independent units
trust boundary	wraps the WHOLE system	wraps EACH unit
verify	ONE end-to-end eval	per-unit runnable evals
HITL posture	one big gate, held longer	step out one gate at a time → dark factory

Lift the swimlane plans into one coherent spec.

ENGINE · SPECKIT / KIRO / OPENSPEC / BMAD

SKILL · plans-to-coherent-spec

OUT · ONE SPEC · ONE E2E EVAL

Take this fork when the pieces only make sense together — a tightly-coupled system where the trust boundary wraps the whole. The reviewed swimlane plans fold into one coherent, coupled specification — the named SDD frameworks (SpecKit, Kiro, OpenSpec, BMAD) live here — which the engine builds against in one piece, verified by a single end-to-end eval. Because the unit of trust is large, a human holds one big gate and releases it as a unit.

LIFT

Plans → spec

the reviewed swimlane plans fold into one coherent, coupled specification the engine builds in one piece.

COUPLE

One system

the trust boundary wraps the whole — pieces only make sense together, verified as a unit.

BIND EVAL

End-to-end

one eval defines done: sources
→ gold → the serving layer
answers, verified as one pipeline.

GATE

One coherent spec exists, the whole system is its unit of trust, and a single end-to-end eval defines done. Because the unit of trust is large and coupled, a human holds one big gate and releases it as a unit — HITL longer. This path then enters the shared gate at Execution.

Atomic. Self-contained. Engine-agnostic.

ENGINE · TASK-SPEC SKILL

SKILL · task-spec --tracker repo

OUT · tasks/* — EACH WITH AN EVAL

Take this fork for loosely-coupled work that each stands alone. The plans decompose into many atomic, eval-backed task-specs — the trust boundary wraps each unit, not the system, and verification is per-unit. Because the gates are granular, trust is earned one unit at a time: each eval that goes green is a gate you no longer hold by hand. This is the path that reaches the Dark Factory — many small, automatable gates a fleet can close on a schedule.

CUT

Atomize
one indivisible unit per task — the smallest thing that can be built and verified on its own.

DESCRIBE

Tech, not agent
name the work and paths, not the harness agent — that comes next; keep each task readable without the harness.

BIND EVAL

Done = ?
a runnable eval defines done — the model test passes, the gold query returns N, the endpoint responds.

GATE

Every task carries a runnable eval — no eval, not a task yet. The trust boundary wraps each unit; verification is per-unit. Because the gates are granular, you step out one gate at a time as each unit's eval earns trust.
Task-spec is self-contained and vendor-portable — Claude, Codex, Kimi, or manual.

Each Task-Spec becomes one tracked issue.

SKILL · task-specs-to-issues

--tracker linear (github | jira)

OUT · 1 SPEC = 1 ISSUE

When tasks leave Tasking, you optionally register each Task-Spec as exactly ONE tracker issue — its state shadow — via the skill task-specs-to-issues --tracker linear (github | jira also). The 1:1 mapping is the whole mechanic: one spec, one issue, no drift. The tracker is a PLUGGABLE backend behind a two-method adapter — read ready issues / write result (failure-as-comment or green-PR) — so GitHub Issues, Linear, or Jira are interchangeable slots, not commitments. blocked-by links carry the dependency graph: 42 blocked-by 41 puts the build order into the tracker itself, as state the loops can read instead of infer.

This is what makes the board state PERFECT. Only the loops write state — never a human nudging a card. Only a green eval moves an issue to done — never a feeling, never an attempt count. Because every transition is mechanical and eval-gated, the board never lies: what it shows is what is true. A "done" column you can trust is the precondition for the Manager loop dispatching against it unattended.

Repo-vs-tracker is a real choice, not an upgrade. Stay at task-spec (repo files) when you want repo-local control — the specs travel with the code, version with the commit, need no external system. Add task-specs-to-issues when you want a board the loops read across sessions and machines — when the Manager runs on a schedule, fans out parallel Workers, and needs shared memory that outlives any one process. The tracker is that shared memory; the repo files are the source.

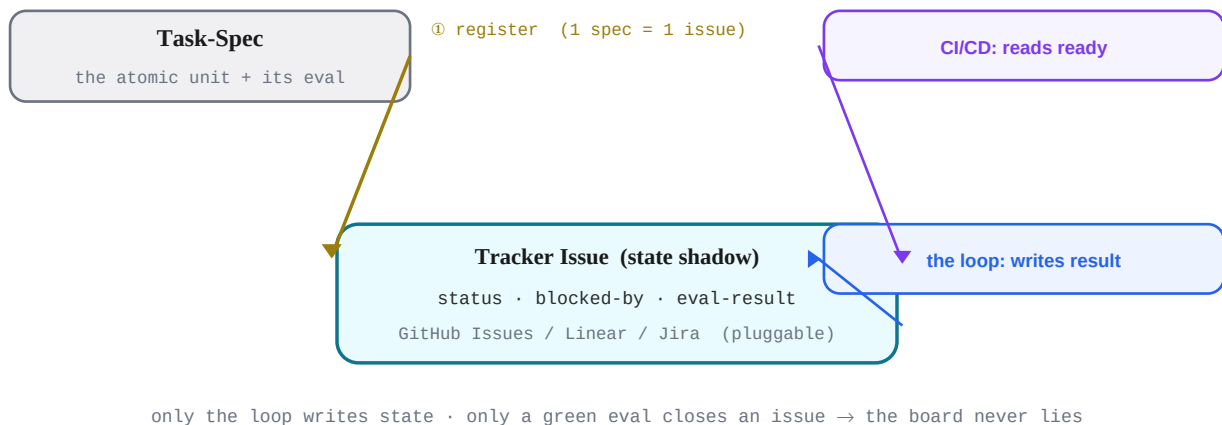


FIG 4 The tracker is the shared state both loops read and write — one Task-Spec registers as one issue; CI/CD reads ready issues, the loop writes results. Only a green eval closes an issue.

Build the rails the work rides on — the quality moat.

ENGINE · AGENTS-KBS-TECH-STACK SKILL

SKILL · stack-to-harness

OUT · .claude/ + AGENTS.md

Pass 6 binds the harness to the SYSTEM'S TECH STACK — not to the two tasks in flight. The dbt knowledge base, the MCP server agent, the DuckDB ruleset exist because the system runs dbt, MCP, and DuckDB — not because ISSUE-41 or ISSUE-42 named them. The tasks SCOPE which stack components are in play, so you scaffold only what the system actually uses, never speculatively. The stack FILLS the harness; the tasks point at it. Tasking precedes Harness for exactly this reason: the work reveals which slice of the stack is in scope.

SCOPE

Read stack

the tasks reveal which techs are in play (dbt + duckdb + mcp) — scaffold exactly those, not speculatively.

SCAFFOLD

Agents + KBs

paired architect + developer per tech, grounded in real docs, plus KBs and rules.

EMIT

Cross-tool

AGENTS.md · Cursor · Copilot — every engine inherits one contract.

GATE

The control plane stands; agents are grounded. The harness is grounded in the system's tech stack — the tasks scope which components are in play, the stack fills the content. **This is the quality moat:** the files that make any commodity agent succeed, and the one artifact that outlives the project.

Build the execution loop. Schedule the Manager later.

Loop Engineering is the June 2026 discipline named by Addy Osmani (with Boris Cherny on Claude Code and Peter Steinberger): you stop prompting agents and start designing the loops that prompt them. The unit of work is no longer the message but the cycle. One primitive recurs everywhere — take an action, read the feedback the environment returns, decide the next move, repeat until the GOAL CONDITION holds. Convergence is defined by the goal, never by attempt-count. A loop that stops because it tried three times has not converged; a loop that stops because the eval is green has.

PASS 8 · THE LOOP — task-loop (the execution loop). This is the loop you build. It takes ONE issue (--issue N, handed to it by a human or by CI), reads its task-spec plus the ADRs and the grounded harness, cuts a branch, and acts — it writes the code. Then it runs the task's own eval. RED: it feeds the failure back into itself and revises, a tight LOCAL loop that repeats until the eval clears. GREEN: it opens a Pull Request that closes the issue. Gate: the task's eval is green — none by hand, none by attempt-count. That green-eval-closes-the-issue discipline is the Dark Factory, one issue at a time.

The Manager is a future CI/CD pass, not an in-session skill. Deciding WHICH issue runs, WHEN, how many in PARALLEL, watching PRs, settling the dependency graph and unblocking dependents — that orchestration layer is a separate concern, and in a Git-native world the platform already provides most of it: GitHub Actions is the scheduler, the pull-request mechanism is the state settlement, branch protection is the gate. So Converge builds the execution loop now (task-loop) and configures the Manager around it later in CI/CD, rather than hand-rolling an in-session orchestrator. You do not need a bespoke Manager to start removing yourself — you need one green-eval loop and a place to schedule it.

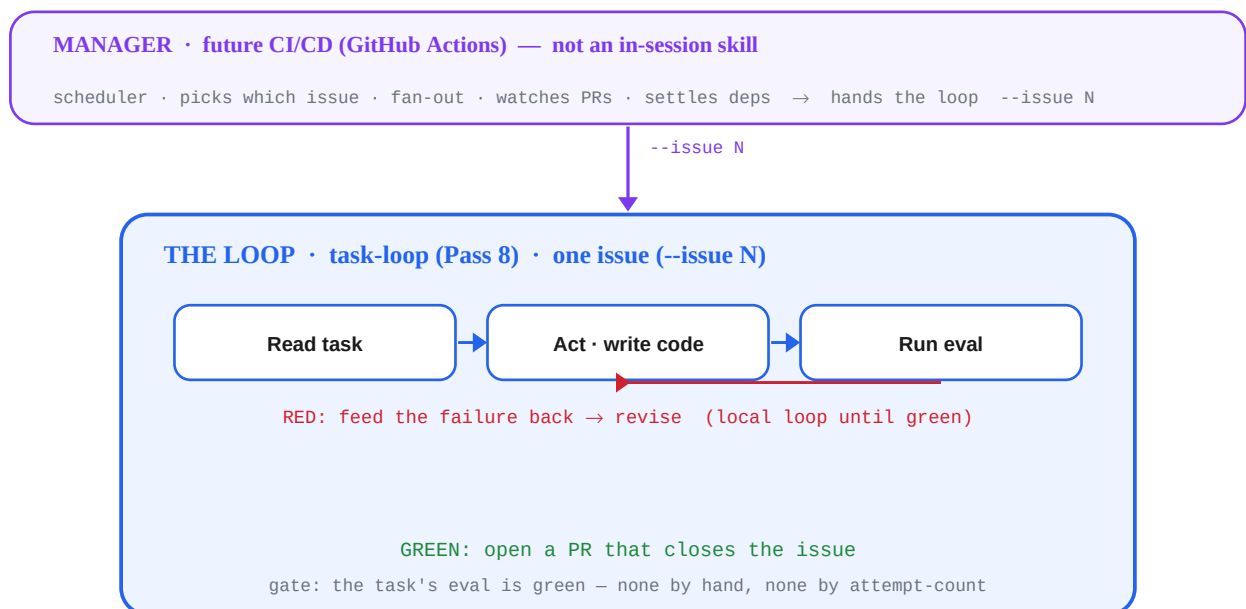


FIG 5 The execution loop — task-loop (Pass 8) takes one issue, writes code, runs the task's eval, feeds red back until green, then opens a PR. The Manager that picks which issue and fans out is a future CI/CD concern (GitHub Actions), not

an in-session skill.

GATE

Pass 8 — the task's eval is green; every issue closed by its OWN eval, none by hand, none by attempt-count. Schedule this loop in CI/CD and the human steps out: that is the Dark Factory, one issue at a time.

Why this is defensible

The two moats

Converge holds two moats, not one, and the discipline is to keep them separate. The Task-Spec is the optionality moat: because the eval — not the agent — defines done, the coding agent becomes a commodity slot you can arbitrage. Swap Codex, Kimi, or Claude freely; the gate is unmoved. The Harness is the quality moat: the control-plane files that make any commodity agent actually succeed. These are not downloadable. They are built, tested, failed, and rebuilt over thousands of hours. The 2026 finding holds — one harness re-run across five models yielded +2.3 to +10.1 points with no retraining. Together: run on commodity models, produce defensible output.

OPTIONALITY

Task-Spec

the eval — not the agent — defines done, so you swap Codex / Kimi / Claude freely; the coding agent is a commodity slot.

QUALITY

Harness

the control-plane files that make any commodity agent succeed; not downloadable, built over thousands of hours.

TOGETHER

Defensible

run on commodity models, produce defensible output. Two moats — optionality and quality — not one.

Positioning vs the field

The canonical 2026 SDD pipeline is Specify -> Plan -> Tasks -> Implement, punctuated by human checkpoints — Spec Kit, Kiro, OpenSpec, BMAD all share this shape. Converge is a loop-closed superset. It adds the three things those frameworks lack: the adversarial Consensus pass, where a different model attacks each plan; the Fork, which decides where the trust boundary sits; and the closed Loop of Pass 8, which drives a whole fleet to green. The named frameworks stop at Implement with a human standing at the checkpoint. Converge keeps going.

THE HONEST BOUNDARY

The Loop converges to green evals, not to correct outcomes. A passing spec test guarantees only that the code matches the spec — if the spec is wrong, the code faithfully implements the wrong thing. The dark factory automates the build, not the judgment of whether the spec was right. The defense is structural and partial, not total: Pass 1's gate forces the spec to answer the brief, and Pass 4's adversary attacks it before any code is written. State this plainly. The intellectual honesty is the point.

One method, every engineer

One method, every engineer. The loop never inspects what a task IS — it inspects only one thing: does the task carry a runnable eval, and is that eval green. All role-specificity is pushed down into the per-task eval. The loop primitive — take an action, read the environment, decide, repeat until the goal condition holds — is identical whether the artifact is a SQL model, a service, an IaC module, or a retriever. That is precisely why Converge is canonical across disciplines: the role does not change the machine, it only changes the assertion the machine waits on.

ROLE	WHAT ITS TASK'S EVAL IS
Data Engineer	a dbt test + a control-sum query against gold
Software Engineer	a unit / integration test that passes
DevOps	terraform plan clean + a smoke test
AI Engineer	a retrieval-precision or eval-set threshold met

THE PRINCIPLE

The role lives in the eval, not the loop. **task-loop** runs whatever until its eval is green — and CI/CD schedules it — both role-agnostic by construction. That is why one method serves Data Engineering, Software, DevOps, and AI Engineering alike.

The method, end to end on one feature

One feature carries the whole framework: "daily revenue per product category, served via MCP so a non-engineer can ask for it." The real metric is `SUM(payments.amount) WHERE payments.status='paid'`, grouped by `products.category` and `date(orders.ordered_at)` in UTC. Here is every pass on the real repo — postgres -> duckdb -> dbt -> MCP.

P1 · Intent	brd-docs-to-tech-req. The BRD becomes a tech-spec: one gold model plus one MCP tool over raw orders + payments + products. Gate: the spec answers the brief.
P2 · Structure	tech-req-to-adrs. Brownfield. Open <code>src/db/01_schema.sql</code> , then write three ADRs: 0001-join-payments-on-order-id, 0002-utc-date-grain, 0003-revenue-is-paid-only. System is now understood versus reality.
P3 · Decompose	reqs-to-swimlane-plans (reads the ADRs). Two plans, plan-altitude only: Plan A transform (silver -> <code>gold_daily_revenue_by_category</code>), Plan B serve (mcp tool <code>query_daily_revenue</code>).
P4 · Consensus	sketch-plans-adversarial-review --adversary codex. A different model attacks: "midnight off-by-one (no TZ)? refunds counted?" Fix: pin UTC (already ADR-0002). Accept: refunds out of v1. THE FORK -> task-driven: two loosely-coupled, independently testable units.
P5 · Task-spec	task-spec --tracker repo. <code>tasks/build_gold_revenue</code> (eval: dbt test + control-sum matches) and <code>tasks/build_mcp_revenue</code> (eval: tool result == gold query). Each task is born with its eval.
① Register	task-specs-to-issues --tracker linear. <code>build_gold_revenue</code> -> ISSUE-41 [ready]; <code>build_mcp_revenue</code> -> ISSUE-42 [blocked-by 41]. The board is now shared state.
P6 · Harness	stack-to-harness. Stack = dbt + duckdb + mcp -> scaffold <code>.claude/</code> with <code>dbt-architect</code> , <code>mcp-developer</code> , KBs, <code>AGENTS.md</code> . The control plane stands. This is the quality moat.
P8 · The Loop	task-loop on GitHub Actions. tick 1: read board -> 41 ready, 42 blocked; dispatch <code>task-loop --issue 41</code> . Worker writes the gold model; dbt test RED (null categories); feedback -> add COALESCE; dbt test GREEN + control-sum matches; PR opened; 41 done. Settle unblocks 42. tick 2: dispatch <code>task-loop --issue 42</code> ; eval green (tool == gold query); PR; 42 done. tick 3: no ready issues -> report "fleet green." Now a non-engineer asks the MCP, "revenue by category yesterday?" — and it answers. Built brief -> deploy, every step eval-verified, the human walked out at the gate. That last step is the Dark Factory, running on this repo.

THE DARK FACTORY, ON THIS REPO

A non-engineer asks the MCP "revenue by category yesterday?" and it answers — built brief → deploy, every step eval-verified, the human walked out at the gate. That last step is the Dark Factory, running on this repo.